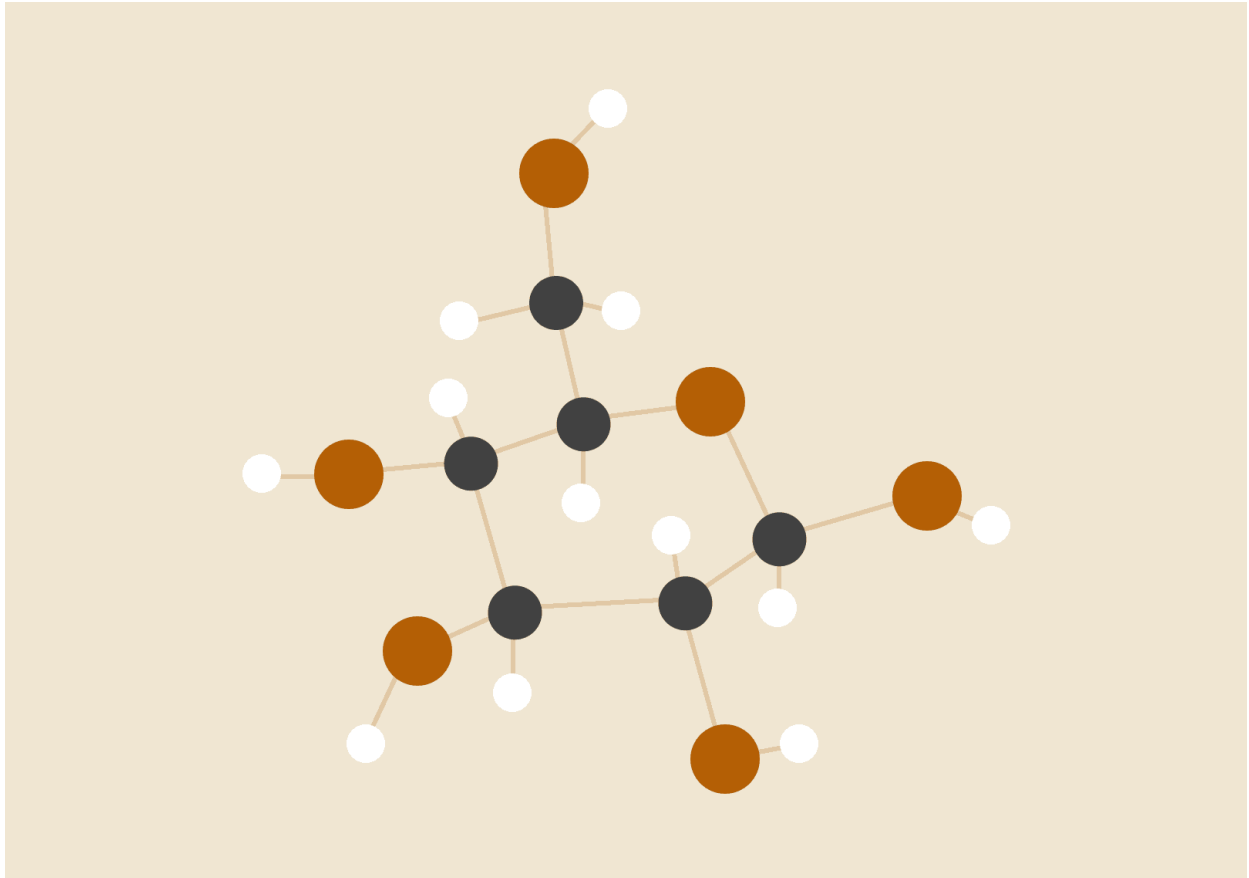


Trabajo Práctico 1: Scheduling



Ariel Eduardo Schenone Duarte

2do Cuatrimestre 2025

Sistemas Operativos

INTRODUCCIÓN

El propósito de este trabajo práctico es analizar y comprender cómo funciona un scheduler de un sistema operativo. El scheduler es un componente muy importante dentro de un sistema operativo ya que es el que decide qué proceso se ejecutará a continuación, siendo vital para una utilización óptima del sistema operativo.

Nos centraremos específicamente en un *scheduler* que usa el algoritmo de *Multilevel Feedback Queue (MLFQ)* el cual es una extensión de *Multilevel Queue*. Su principal característica es que existen varias colas de *ready* con una cierta cantidad de *quantum*, separadas por categorías de procesos (prioridades), cuya gran diferencia con respecto al algoritmo de *Multilevel Queue* es que los procesos pueden cambiar de cola.

Para familiarizarnos con su comportamiento utilizaremos un simulador y generaremos distintos problemas para observar su lógica.

EJERCICIO 1

REGLAS DEL SIMULADOR

- Número de colas: 3 prioridades.
- Quantum de tiempo:
 - Prioridad 2 con 4 quantum.
 - Prioridad 1 con 8 quantum.
 - Prioridad 0 con 10 de quantum.
- Tareas por nivel:
 - Prioridad 2 con allotment 1.
 - Prioridad 1 con allotment 2.
 - Prioridad 0 con allotment 10.
- E/S: Deshabilitado.
- Tiempo máximo de duración de una tarea: 30ms.
- Semilla: 140925

DATOS DEL PROBLEMA 1

Proceso 0: Llega en $t=0$, requiere 9 ms de CPU.

Proceso 1: Llega en $t=0$, requiere 18 ms de CPU.

Proceso 2: Llega en $t=0$, requiere 21 ms de CPU.

Comando a ejecutar: `python3 mlfq.py -c -Q 4,8,10 -A 1,2,10 -j 3 -M 0 -m 30 -s 140925`

HIPÓTESIS

Dado que el scheduler está configurado para usar una política de quantums cortos en los niveles de mayor prioridad, se espera que el proceso que requiere menos CPU (Proceso 0) tenga un **turnaround** significativamente menor que los procesos que requieren de más CPU (Proceso 1 y Proceso 2). Esto ocurre porque los procesos cortos completan su trabajo en las colas de alta prioridad, mientras que los procesos más largos serán degradados a colas de menor prioridad y recibirán servicio más espaciado, aumentando su tiempo de espera y por ende su turnaround. Además el tiempo de respuesta de los procesos irá en incremento debido a que todos los procesos ingresan al mismo tiempo ($t=0$), pero deben esperar su turno de ejecución asignado por el scheduler.

RESULTADOS

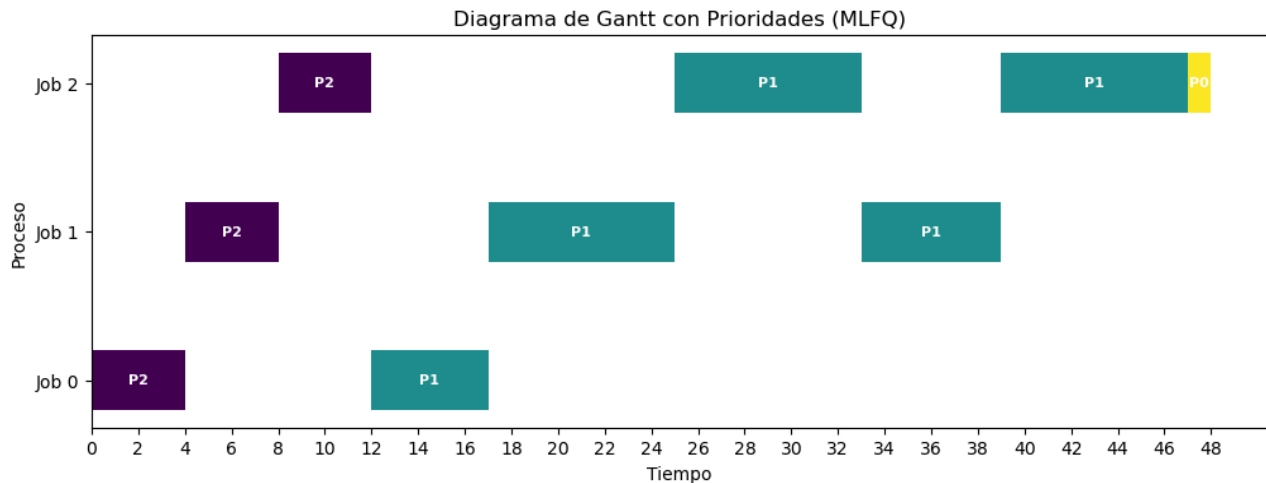
Basado en los datos de entrada del problema 1 y luego de la simulación podemos ver las estadísticas finales.

```
Job  0: startTime    0 - response    0 - turnaround   17
Job  1: startTime    0 - response    4 - turnaround   39
Job  2: startTime    0 - response    8 - turnaround   48
```

CONCLUSIÓN

Como podemos observar, los resultados confirman las hipótesis iniciales. Las estadísticas nos muestran que la diferencia entre el **turnaround** del proceso 0, el cual requería de menos tiempo de CPU, es bastante menor en comparación con el resto de procesos que necesitaban de más tiempo de CPU. Mientras que el tiempo de espera creció de manera escalonada dado que todos los procesos ingresaron simultáneamente.

Esto demuestra que el scheduler cumplió con su objetivo de favorecer a los procesos cortos, que completaron rápidamente su ejecución, mientras que los procesos más largos fueron degradados a niveles inferiores de prioridad y experimentaron mayores tiempos de espera.



DATOS DEL PROBLEMA 2

Proceso 0: Llega en $t=0$, requiere 13 ms de CPU.

Proceso 1: Llega en $t=0$, requiere 23 ms de CPU.

Proceso 2: Llega en $t=0$, requiere 9 ms de CPU.

Proceso 3: Llega en $t=0$, requiere 22 ms de CPU.

Comando a ejecutar: `python3 mlfq.py -c -j 4 -m 30 -M 0 -s 651910 -q 16`

HIPÓTESIS

Se espera que, al configurar un quantum uniforme y elevado en todos los niveles de prioridad, el tiempo de espera de los procesos aumente significativamente, ya que todos ingresan en $t=0$ y los últimos en ser seleccionados deberán esperar a que termine un quantum muy largo del proceso anterior antes de acceder a CPU. Esto también afectará al tiempo de respuesta, que crecerá de manera más marcada entre procesos. En consecuencia, los procesos cortos no obtendrán la ventaja habitual de completar

rápidamente, lo que reduce la capacidad del scheduler de favorecer a tareas interactivas.

RESULTADOS

Basado en los datos de entrada del problema 2 y luego de la simulación podemos ver las estadísticas finales.

Job 0: startTime 0 - response 0 - turnaround 13

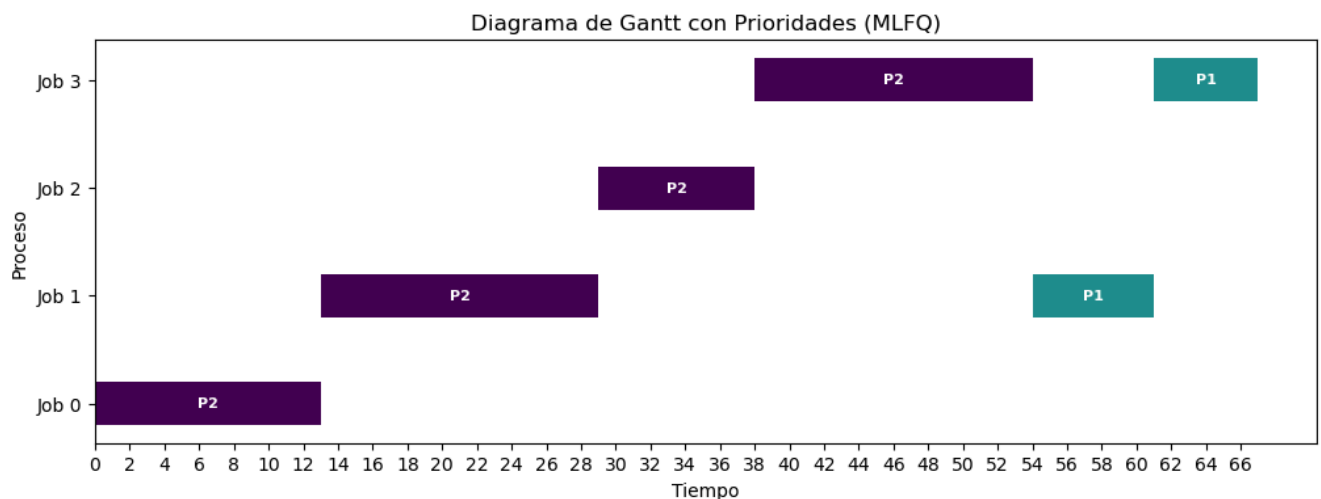
Job 1: startTime 0 - response 13 - turnaround 61

Job 2: startTime 0 - response 29 - turnaround 38

Job 3: startTime 0 - response 38 - turnaround 67

CONCLUSIÓN

Los resultados confirman la hipótesis inicial. La métrica de *response* muestra un incremento escalonado entre los procesos: **P0 = 0 ms, P1 = 13 ms, P2 = 29 ms, P3 = 38 ms** con una diferencia media entre procesos de más de **10 ms**. Esto evidencia que, al ingresar todos los procesos en $t = 0$ y con la política usada, los procesos que no son atendidos inmediatamente sufren un retraso significativo en su primer acceso al CPU. Además, se observa un efecto claro sobre el *turnaround*: un proceso relativamente corto (9 ms de CPU) puede tardar **38 ms en completarse**, porque fue desplazado por otros procesos que ocuparon la CPU durante quanta largos.



EJERCICIO 2

INTRODUCCIÓN

Para este ejercicio se nos piden agregar unas funcionalidades al código del simulador, específicamente métricas de **throughput** y el **waiting time**. Para lograr incorporar estas métricas primero debemos analizar y comprender que mide cada una, que representa en el scheduler y cómo se mide a partir de las métricas que están disponibles. A continuación se presenta la definición de cada una de las métricas.

DEFINICIÓN DE MÉTRICAS

THROUGHPUT

Definición: Es la cantidad de procesos que el sistema logra completar en una determinada unidad de tiempo.

Interpretación: Mide la *productividad* del scheduler, es decir, qué tan eficiente es en terminar procesos a lo largo del tiempo.

Fórmula:

$$throughput(t) = \frac{\text{Números de procesos finalizados hasta el tiempo } t}{t}$$

Observación: Un throughput alto indica que el sistema termina procesos de forma más rápida; sin embargo, no garantiza tiempos de respuesta bajos ni equidad entre procesos.

WAITING TIME

Definición: Es el tiempo total que un proceso pasa en la cola de *ready* esperando para ser ejecutado en CPU.

Interpretación: Refleja la *latencia* experimentada por cada proceso; un valor alto implica que el proceso pasó mucho tiempo esperando, lo cual puede ser crítico para procesos interactivos.

Fórmula:

$$waiting_i = turnaround_i - burst_i - IO\ Time_i$$

Donde:

- $Turnaround_i = Finish_i - Start_i$
- $Burst_i$ Es el tiempo total de CPU requerido por el proceso.
- $IO\ Time_i$ Es el tiempo total en I/O.

MODIFICACIONES DEL SIMULADOR

THROUGHPUT

Para implementar la métrica de *throughput*, utilizaremos las variables `finishedJobs` donde se van guardando la cantidad de trabajos que terminaron de ejecutarse y `currTime` donde se va guardando el tiempo actual. Para calcularlo dentro del bucle dividimos la variable de trabajos terminados por el tiempo actual.

Con el fin de observar cómo evoluciona esta métrica, decidimos imprimir su valor cada dos unidades de tiempo. Para ello, verificamos si el tiempo actual es par y, en ese caso, mostramos el throughput junto con la cantidad de procesos terminados.

```
.....  
  
# THROUGHPUT  
  
throughput = float(finishedJobs) / float(currTime)  
  
if currTime % 2 == 0:  
    print('\n [ time %d ] THROUGHPUT %.2f ( finished %d jobs  
out of %d )\n' % (currTime, throughput, finishedJobs, numJobs))  
  
.....
```

Finalmente, también incluimos el cálculo de throughput en el bloque de estadísticas finales.

```
# print out statistics

print('')

print('Final statistics:')

responseSum = 0

turnaroundSum = 0

throughput = float(finishedJobs) / float(currTime)

for i in range(numJobs):

    response = job[i]['firstRun'] - job[i]['startTime']

    turnaround = job[i]['endTime'] - job[i]['startTime']

    print('  Job %2d: startTime %3d - response %3d - turnaround %3d'
          % (i, job[i]['startTime'], response, turnaround))

    responseSum += response

    turnaroundSum += turnaround

print('\n  Avg %2d: startTime n/a - response %.2f - turnaround %.2f
- throughput %.2f' % (i, float(responseSum)/numJobs,
float(turnaroundSum)/numJobs, throughput))

print('\n')
```

WAITING TIME

Para implementar la métrica de *waiting time* utilizaremos una métrica ya calculada por el simulador (*turnaround*) y un dato de cada trabajo almacenado en la estructura de datos `job[i]` -> `runTime`, el cual representa el tiempo total de CPU requerido por el trabajo *i*.

Además tuve que agregar otro dato almacenado en la estructura de datos `job` que es el `totalIOTime` donde se guarda el tiempo total de E/S del proceso.

```
job[jobCnt] = {'currPri':hiQueue, 'ticksLeft':quantum[hiQueue],
               'allotLeft':allotment[hiQueue], 'startTime':startTime,
               'runTime':runTime, 'timeLeft':runTime,
               'ioFreq':ioFreq, 'doingIO':False,
               'firstRun':-1, 'totalIOTime':0}
```

Luego esa variable se modifica luego de aumentar el `currTime`, donde recorro todos los trabajos y me fijo cuales están haciendo alguna operación de IO y le sumo 1 a ese campo.

```
.....
    # UPDATE TIME
    currTime += 1

    for j in job:
        if job[j]['doingIO'] == True:
            job[j]['totalIOTime'] += 1
.....
```

En primer lugar, declaramos la variable `waitingSum`, donde iremos acumulando los tiempos de espera de cada trabajo, con el fin de calcular posteriormente el promedio sobre todos los trabajos:

Luego, dentro del bucle que recorre todos los trabajos, declaramos la variable `waiting`, en la cual guardaremos el tiempo de espera de cada trabajo. Su valor se obtiene como la diferencia entre el *turnaround* del trabajo, su `runTime` y su tiempo total de E/S.

Finalmente, imprimimos el valor de `waiting` para cada trabajo y lo acumulamos en `waitingSum` para calcular el promedio total.

```
# print out statistics

print('')

print('Final statistics:')

responseSum = 0

turnaroundSum = 0
```

```

throughput = float(finishedJobs) / float(currTime)

waitingSum = 0

for i in range(numJobs):

    response = job[i]['firstRun'] - job[i]['startTime']

    turnaround = job[i]['endTime'] - job[i]['startTime']

    waiting = turnaround - job[i]['runTime'] - job[i]['totalIOTime']

    print(' Job %2d: startTime %3d - response %3d - turnaround %3d
- waiting %3d' % (i, job[i]['startTime'], response, turnaround,
waiting))

    responseSum += response

    turnaroundSum += turnaround

    waitingSum += waiting

print('\n Avg %2d: startTime n/a - response %.2f - turnaround %.2f
- waiting %.2f - throughput %.2f' % (i, float(responseSum)/numJobs,
float(turnaroundSum)/numJobs, float(waitingSum)/numJobs,
throughput))

print('\n')

```

EXPERIMENTACIÓN

DATOS DEL PROBLEMA

Proceso 0: Llega en t=0, requiere 10 ms de CPU, hace entrada salida cada 2 ms.

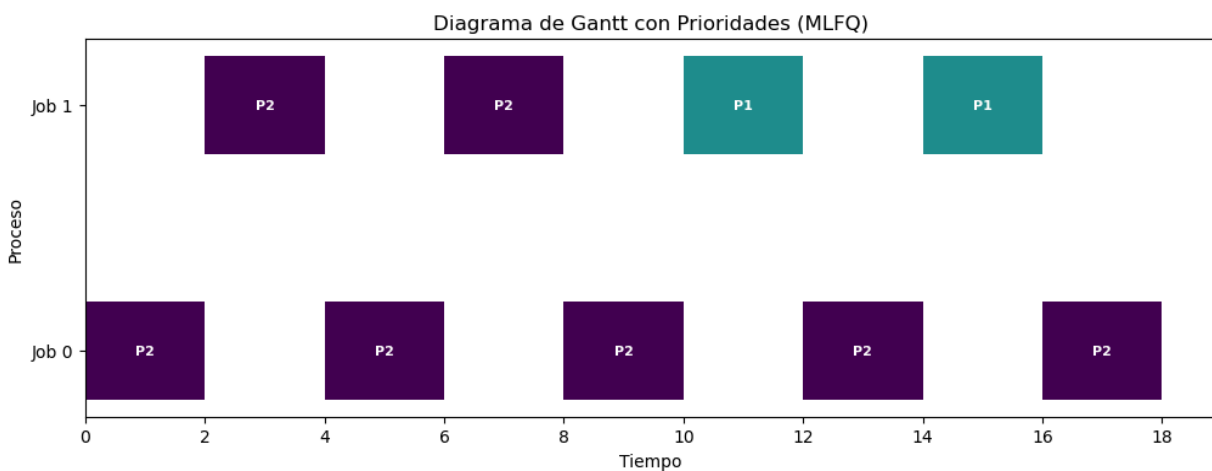
Proceso 1: Llega en t=0, requiere 8 ms de CPU, no hace entrada salida.

Comando a ejecutar: python3 mlfq.py -j 2 -l 0,10,2:0,8,0 -Q 4,8,12 -i 2
-S -I -c.

Una configuración especial del simulador es la opción **-I**, que permite que los procesos que regresan de una operación de entrada/salida vayan directamente al principio de la cola.

Otra opción relevante es **-S**, que evita que los procesos que realizan operaciones de entrada/salida pierdan prioridad y se mantengan en la más alta prioridad.

En esta prueba podemos distinguir que el proceso I/O-bound (**Job 0**) alterna constantemente entre CPU y operaciones de entrada/salida, mientras que el **Job 1** solo ejecuta CPU.



Podemos observar que el **Job 0** no espera en ningún momento: cuando termina una operación de entrada/salida, pasa directamente a ejecutarse, desplazando así al Job 1, que se mantiene en estado *ready*, esperando su turno.

Por lo tanto, el **Job 0** no tiene tiempo de espera, mientras que el **Job 1** sí acumula tiempo de espera.

RESULTADOS

Final statistics:

Job 0: startTime 0 - response 0 - turnaround 18 - waiting 0

Job 1: startTime 0 - response 2 - turnaround 16 - waiting 8

El Job 0 se ve claramente favorecido por las opciones **-I** y **-S**, ya que siempre vuelve al frente de la cola y mantiene su prioridad tras cada operación de entrada/salida.

VERIFICACIÓN DE THROUGHPUT

Para verificar que la implementación del cálculo de *throughput* funciona correctamente, se ejecutó el mismo comando anterior.

Comando a ejecutar: `python3 mlfq.py -j 2 -l 0,10,2:0,8,0 -Q 4,8,12 -i 2 -S -I -c.`

Durante la ejecución, el simulador muestra el *throughput* cada cierto 2 ms.

```
...
[ time 2 ] THROUGHPUT 0.00 ( finished 0 jobs out of 2 )
...
[ time 4 ] THROUGHPUT 0.00 ( finished 0 jobs out of 2 )
...
[ time 6 ] THROUGHPUT 0.00 ( finished 0 jobs out of 2 )
...
[ time 16 ] THROUGHPUT 0.00 ( finished 0 jobs out of 2 )
[ time 16 ] FINISHED JOB 1
[ time 16 ] THROUGHPUT 0.06 ( finished 1 jobs out of 2 )
...
[ time 18 ] THROUGHPUT 0.06 ( finished 1 jobs out of 2 )
[ time 18 ] FINISHED JOB 0
[ time 18 ] THROUGHPUT 0.11 ( finished 2 jobs out of 2 )
```

Podemos observar que el simulador muestra el *throughput* en intervalos de tiempo (cada 2 ms) y a su vez inmediatamente después de la finalización de algún trabajo.

Esto nos permite distinguir el impacto que tuvo en la métrica la finalización de algún trabajo.

Se observa que el *throughput* no se modifica hasta el tiempo 16, cuando el **Job 1** termina.

En ese momento se recalcula, obteniendo un valor de 0.06 , que resulta de dividir 1 trabajo finalizado entre el tiempo total transcurrido (16 ms).

El mismo comportamiento se repite en el tiempo 18, donde el valor pasa de 0.06 a 0.11 al terminar el segundo proceso.

CONCLUSIÓN

Al finalizar la simulación se observa el reporte promedio final de las métricas.

Avg 1: startTime n/a - response 1.00 - turnaround 17.00 - waiting 4.00 - throughput 0.11.

El valor promedio de *throughput* (0.11) coincide con los cálculos observados durante la simulación, reflejando que dos trabajos fueron completados en un tiempo total de 18 ms. Y a su vez también coincide el promedio del *waiting*, donde suma el *waiting* de Job 0 (0) y el *waiting* del Job 1 (8) y lo divide por la cantidad de trabajos, en este caso 2, lo que da el resultado de 4 en promedio.

Este resultado confirma que la implementación del cálculo de *throughput* y *waiting* es correcta.

EJERCICIO 3

CONFIGURACIÓN DEL SCHEDULER

Para este ejercicio se nos pide simular un escenario con 2 trabajos, de los cuales uno utiliza operaciones de E/S para aprovecharse del scheduler y acaparar recursos del CPU.

Una configuración del scheduler que favorece esta situación es habilitar la opción en la cual los trabajos que terminan una operación de E/S pasan al frente de la cola. De esta manera, si un trabajo se interrumpe para realizar E/S mientras otro se está ejecutando, al regresar de la operación I/O será colocado inmediatamente al principio de la cola, garantizando que sea el próximo en ejecutarse.

Otra modificación posible es impedir que los procesos con ráfagas de E/S bajen de prioridad. Combinándolo con el reingreso al frente de la cola, hace que el proceso I/O-bound reciba repetidamente el CPU con mínima espera, relegando al proceso puramente CPU-bound.

Siguiendo con las modificaciones del scheduler, podemos cambiar el tamaño de los

quantums: en los niveles de mayor prioridad definir quantums cortos, mientras que en los demás niveles de menor prioridad utilizar quantums más largos. Esta configuración incrementa la ventaja de los procesos I/O bound, ya que suelen necesitar poco tiempo de CPU y, al completarlas sin agotar el quantum, se mantienen en las colas superiores.

Entre otras modificaciones: es que la duración fija de 1ms de una operación E/S para los procesos.

DATOS DEL PROBLEMA

Proceso 0: Llega en $t=0$, requiere 50 ms de CPU, frecuencia de E/S: 2ms.

Proceso 1: Llega en $t=0$, requiere 28 ms de CPU, sin E/S.

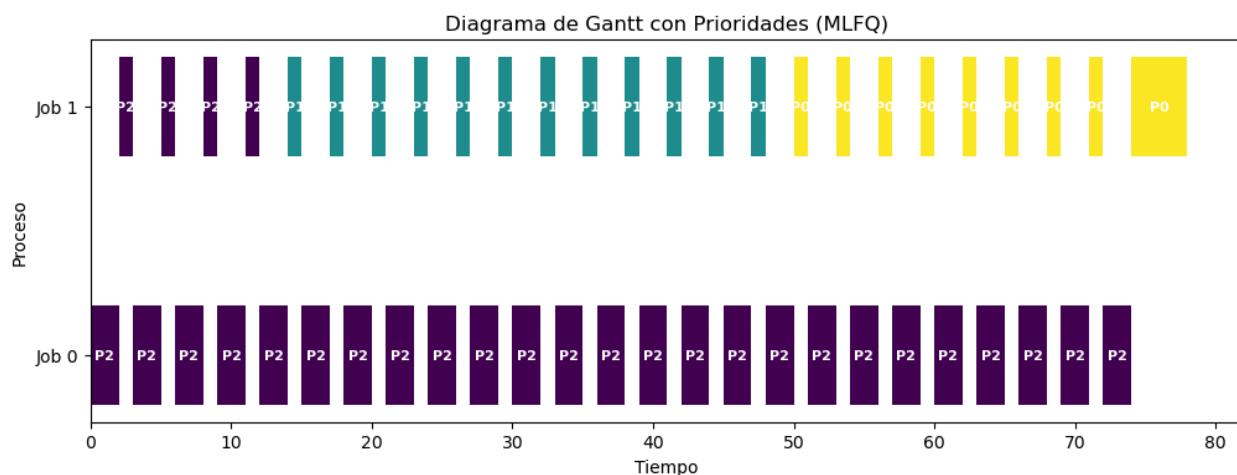
Comando a ejecutar: `python3 mlfq.py -c -l 0,50,2:0,28,0 -Q 4,12,20 -S -I -i 1`

RESULTADO

Basado en los datos de entrada del problema y luego de la simulación podemos ver las estadísticas finales.

```
Job 0: startTime 0 - response 0 - turnaround 74 - waiting 0
```

```
Job 1: startTime 0 - response 2 - turnaround 78 - waiting 50
```



CONCLUSIÓN

Basándonos en los resultados obtenidos podemos decir que este es un escenario donde existe un proceso que acapara el uso del CPU aprovechándose del mecanismo de E/S , relegando al proceso que solo hace operaciones de CPU.

Observamos que el proceso 0 pese a tener el doble de requerimiento de CPU que el proceso 1, termina antes y nunca baja de la prioridad más alta.

En este ejemplo se ve muy clara la diferencia entre el proceso I/O bound y el proceso CPU bound, esta diferencia se incrementa si se cuenta con más procesos que solo requieren de CPU.

EJERCICIO 4

Para este ejercicio debemos hallar dos configuraciones del simulador:

- Una que sea beneficiosa para el trabajo interactivo y perjudicial para el resto
- Y otra que sea perjudicial para el trabajo interactivo y beneficioso para el resto

DATOS DEL PROBLEMA

Proceso 0: Llega en $t=0$, requiere 30 ms de CPU, sin E/S.

Proceso 1: Llega en $t=0$, requiere 8 ms de CPU, pide E/S cada 2ms.

Proceso 2: Llega en $t=10$, requiere de 12 ms de CPU, sin E/S.

CONFIGURACIÓN BENEFICIOSA PARA PROCESOS INTERACTIVOS

Retomando la idea del ejercicio anterior, podemos utilizar las mismas configuraciones

- No permitir que los procesos que hacen operaciones de E/S bajen de prioridad
- Los procesos que terminan su operaciones de E/S reingresen primero a la cola
- Quantums cortos en el nivel de mayor prioridad (4ms) , y quantums más largos en los de menor prioridad (14ms, 20ms)
- Duración fija de las operaciones de E/S: 1 ms

Comando a ejecutar: `python3 mlfq.py -c -l 0,30,0:0,8,2:10,10,0 -I -S -i 1 -Q 4,14,20`

RESULTADOS

Job 0: startTime 0 - response 0 - turnaround 48 - waiting 18

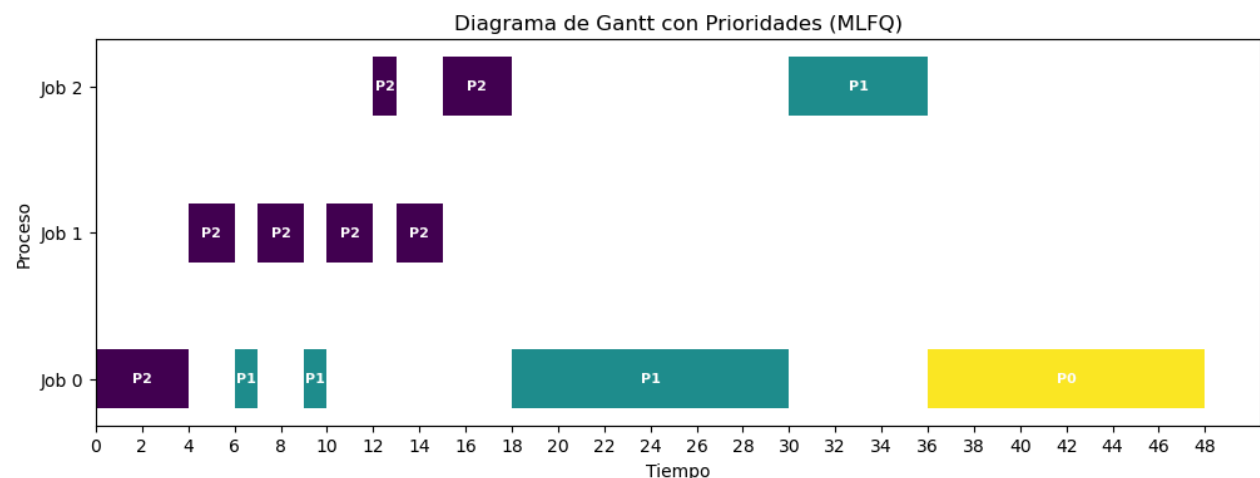
Job 1: startTime 0 - response 4 - turnaround 15 - waiting 4

Job 2: startTime 10 - response 2 - turnaround 26 - waiting 16

CONCLUSIÓN

Con esta configuración podemos observar que el interactivo (proceso 1), se vio beneficiado por esta configuración del simulador, terminando primero su ejecución, a pesar de no haber empezado a ejecutarse primero. También vemos que el tiempo de espera es bastante menor en comparación a los otros procesos.

Se aprecia aún más al ver la diferencia significativa entre el turnaround del proceso 0 y el proceso 1.



CONFIGURACIÓN PERJUDICIAL PARA PROCESOS INTERACTIVOS

Para esta configuración debemos favorecer a los procesos CPU bound tratando de perjudicar a los procesos que hacen muchas operaciones de E/S.

En primer lugar, una buena idea sería tener quantums largos en los niveles de mayor prioridad que favorezcan el uso intensivo del CPU, así evitar que bajen de prioridad o ser interrumpidos.

Otra opción que beneficia a los procesos que requieren mucha CPU es cambiar el

allotment, que mide cuánta quanta puede consumir un proceso en un nivel antes de bajar de prioridad. Mientras más alto es el allotment más tiempo tarda en bajar de prioridad.

Con estas configuración del simulador priorizamos que los procesos estén el máximo tiempo posible en la prioridad más alta, la cual tiene el quantum más alto y puedan usar todo el CPU que requieran por más tiempo sin ser interrumpidos.

RESULTADOS

```
Job 0: startTime 0 - response 0 - turnaround 30 - waiting 0
Job 1: startTime 0 - response 30 - turnaround 58 - waiting 50
Job 2: startTime 10 - response 22 - turnaround 32 - waiting 22
```

CONCLUSIÓN

Podemos observar los resultados obtenidos y verificar que efectivamente el proceso de I/O bound quedó relegado y terminó de ejecutar último debido a la gran cantidad de quantum que permite que los procesos puedan ejecutar bastante tiempo sin ser interrumpidos, mientras que los demás procesos que usaban bastante CPU, terminaron antes con bastante diferencia.

